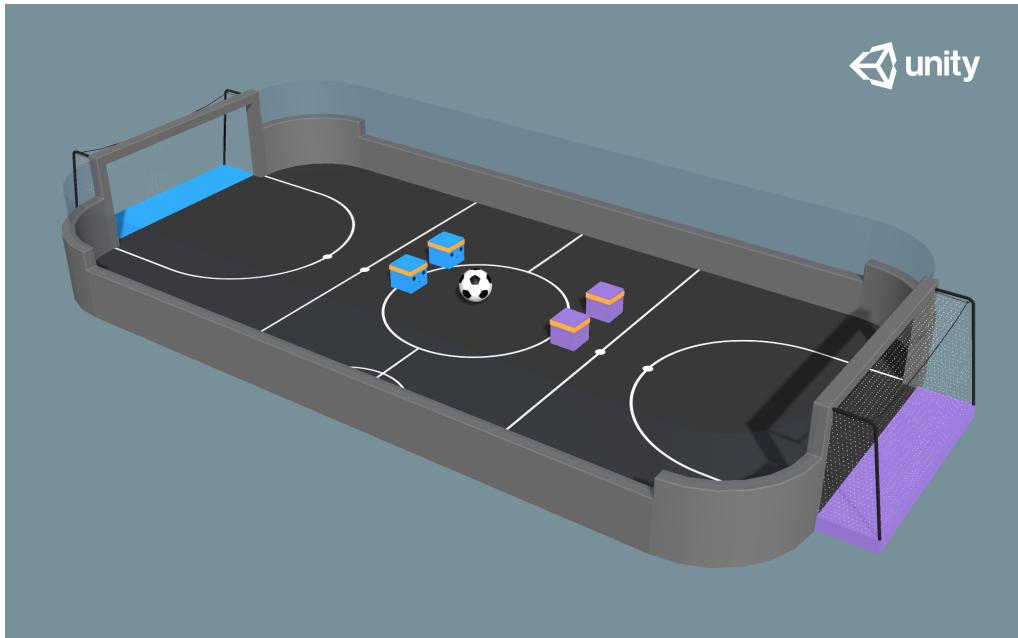


Project Manual - Bachelor Computer Science - Year 2
Project 2.1 - AI and Machine Learning
Machine Learning in the Unity Game Engine

Dr. Dennis Soemers

2026-2027



Period 1.1, 1.2, and 1.3
Academic year 2026-2027
Department of Advanced Computing Sciences
Faculty of Science and Engineering
Maastricht University

1 Project Description

The central topic of Project 2.1 (*Computer Science programme, AI and Machine Learning module*) is to apply Machine Learning (ML) techniques to data collected from a real-time video game engine with AI-controlled agents. Along the way, you will gain experience combining your own code and data with large existing frameworks, libraries, and code and data written and collected by others.

1.1 Background

Unity¹ is an engine for the development of 3D real-time video games, among other types. The Unity Machine Learning Agents Toolkit (ML-Agents) [1] extends the engine with a suite of Deep Reinforcement Learning (DRL) algorithms, as well as example environments in which to test them. Reinforcement Learning (RL) is a branch of ML concerned with training agents, which live in some environment in which they may take sequences of actions over longer time spans, to select their actions so as to optimise sums of “reward” signals. Agents may be tasked with balancing balls on top of themselves, navigating to a goal, playing soccer, and so on. See Figure 1 for screenshots of a selection of example environments that are included in ML-Agents.

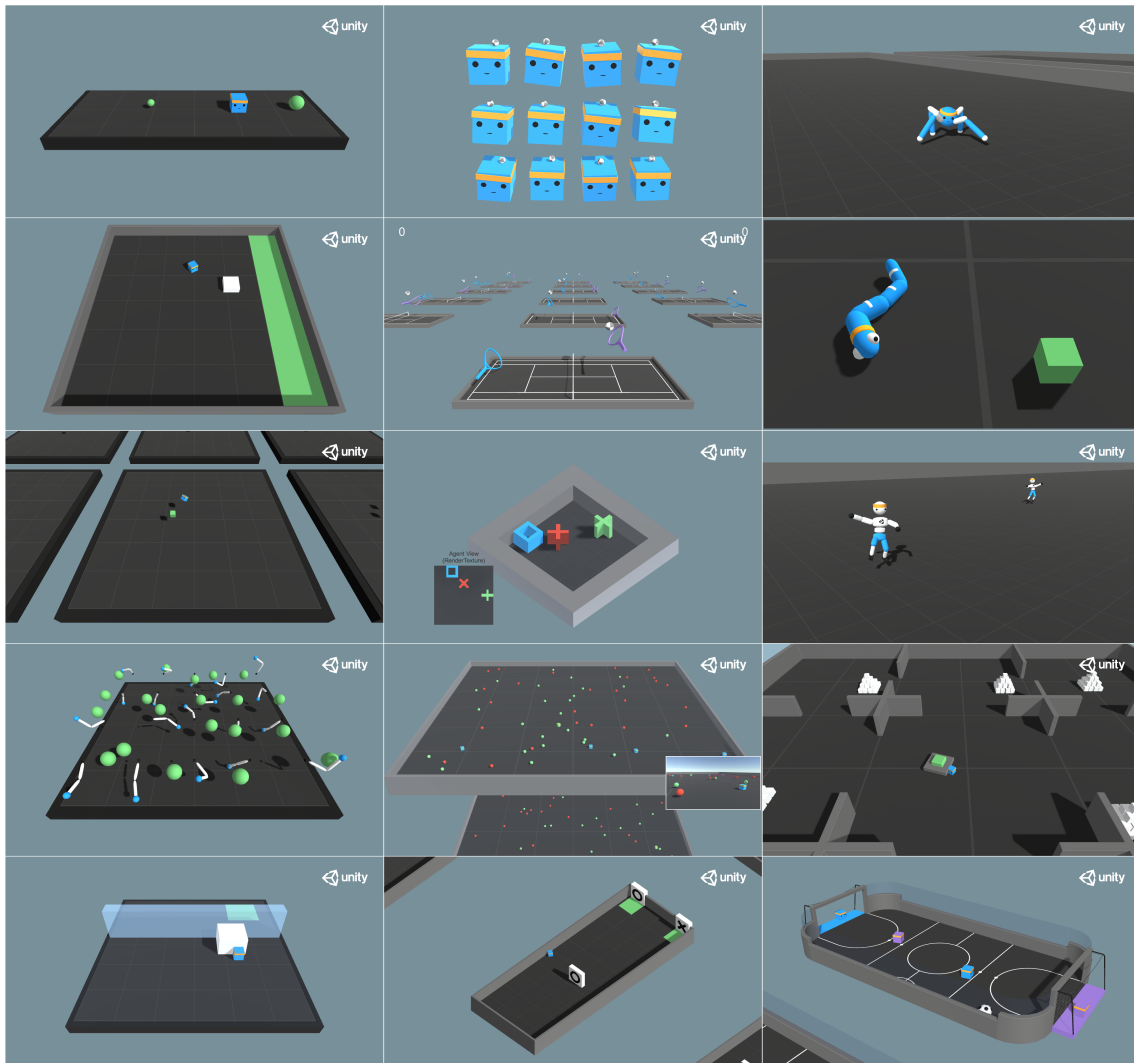


Figure 1: Example environments in Unity ML-Agents. Image source: [1].

¹<https://unity.com/>

While the DRL techniques in ML-Agents are also a form of ML, these algorithms are not the focus of your project. You may treat them as black-box techniques, and use them to train agents that can autonomously play games in the framework, without necessarily diving into their internal details. Your primary focus will be only collecting data from these training processes, and/or the resulting agents and how they perform in the engine, and use this data for (un)supervised ML. You will also have to learn how to effectively use large frameworks and codebases written by other people, possibly using one or more programming languages that you are not yet familiar with. These are important skills to pick up as, in the remainder of your lives as computer scientists and programmers, you will rarely if ever work solely with code that you wrote yourself from scratch.

1.2 Objective

Your main objective is to (i) collect data, and study it using ML techniques, on intelligent agents training and acting in 3D real-time simulations, and (ii) build this in a clean, well-documented, public GitHub repository which you will be able to include in your programming portfolio.

- In the **first phase** of the project, you will focus on making a plan (for the entire semester), ensuring that every member of the group sets up and understands the necessary tools and infrastructure, and figuring out how you can add or edit, compile, and run your own code within Unity and the ML-Agents environments. Your plan is also expected to contain details on what data you plan to collect, and how.
- In the **second phase**, it is expected that your primary focus will be on data collection. This may require code changes or additions inside Unity and/or ML-Agents to record any data of interest that is not already recorded by default. Note that any DRL training processes you may want to run can take a long time, so you should plan ahead, start early, and try to effectively distribute workloads over the group members. You can also already start training ML models on the data.
- Finally, in the **third phase**, we expect that your focus will shift more towards the ML techniques, whilst still continuing to generate more data.

Throughout all three phases, you should ensure to carefully and clearly document your work and provide instructions for any and all used tools and dependencies. The documentation should be written such that, if a new member were to join your team midway through any phase, they would be able to quickly and easily get started.

1.3 Software

For this project, you are required to use:

- Git, for version control, with your repository/repositories hosted on GitHub with public visibility.
- The Unity game engine (the “Unity Personal” free license will suffice, but you may also try to get the “Unity Student” license).
- The Unity Machine Learning Agents Toolkit (ML-Agents) (it is recommended to use the fork and branch from <https://github.com/DennisSoemers/ml-agents/tree/fix-numpy-release-21-branch>, as we know this works fine from previous years; if the particular fix in that branch has since also been merged into the official repository, you can also try to go with that one).

You are free to use any additional libraries or software as you see fit, but *you will be expected to motivate your choice for selecting them*. Unity traditionally uses C# as its

main programming language, and ML-Agents is built using **Python**, which makes it likely you will have to use one or both of these languages. **Python** is also the most commonly used language for ML in general.

2 Project Tasks

The following three subsections describe in more detail what the general expectations are for each phase, and how you will be assessed.

2.1 First phase

Your **deliverables** for the first phase are:

1. **A written project plan**, in which you write a detailed plan for the steps to take over the duration of the project. The plan should include at least a description of the work you aim to do, what kinds of research questions you would like to be able to answer by the end of the project, what data you plan to collect, a rough time schedule, and an assessment of risks and how to work around them. The maximum number of pages is 10, but using fewer pages is also fine.
2. **At least one publicly visible GitHub repository**, with relevant documentation, to contain your work. You will continue iterating on this page throughout the entire project.

Throughout this phase, if you aren't already, you will have to become acquainted with (among other things):

- Using git and GitHub for version control. Every individual member of your group should have a GitHub account, and should understand what the following terms mean in the context of git and GitHub: commit, pull, push, branch, fork.
- Basic use of the Unity game engine. You will have to install it, install the ML-Agents package, open example scenarios from ML-Agents, and play around in them.
 - There is documentation for all this online, but it may not always be perfectly clear or accurate. Please ensure that any installation and setup instructions are 100% clear in your own repository.
- Installation of Python, and packages for it. The ML-Agents package is written in Python, and has many other Python packages as dependencies (in many cases requiring specific versions). For other work (for example, the *Introduction to AI* course), you may also need or want specific versions of Python packages (or Python itself) installed, which may differ from the versions that ML-Agents requires! Therefore, we strongly recommend using *virtual environments*,² such that you may have different versions installed in different virtual environments.

In the first phase, we recommend that you also already try implementing some code changes (not in-editor changes to variables or level layout) to agent, learning, and/or environment code in one or more ML-Agents example environments. For this phase, there is no specific requirement as to *what* change you implement, as long as you can do *something*. The purpose of this is for you to figure out how best to structure your entire project and installation. For example, you will have to figure out whether you can easily implement code changes in a completely separate repository, or whether this is better done directly inside the ML-Agents codebase itself. If the latter is the case, you may prefer starting out with your own GitHub repository being a fork of the official one, rather

²<https://docs.python.org/3/library/venv.html>

than starting from an empty repository. If you do this, you should still completely overhaul and update the documentation that shows on the front page of your repository, such that it reflects your work and how it differs from and is related to the original repository.

2.2 Second phase

Your focus in the second phase is expected to be on **data collection**, as well as an initial exploration of **applying ML techniques** to the data. **Your core objective throughout the entire project** will be to train ML models that can predict properties of deep RL training runs.³ For example:

- Can you predict how long (in wall-clock-time) a training run will take?
- Can you predict how much RAM (peak, average, ...) a training run will use?
- Can you predict how well an agent will be able to perform in a game / task after training?
- Can you predict how deep into a training process (measured in number of steps, or wall-clock-time) an agent will be able to reach a certain relevant threshold level of performance?

Your work should cover at least one of these examples, but may cover multiple, or even new ideas you come up with yourself. To accomplish this task, you will of course have to collect examples of actual training runs (i.e., really run those training runs) and measure the variables you wish to be able to predict via ML. However, you will also need to collect relevant features based on which your models are trained to make their predictions. Examples of such features (again, you may come up with additional ones) include:

- Descriptions of hardware used (available RAM measured in GB, CPU clock speed measured in GHz, number of cores in CPU, ...).
- DRL algorithm used for the training run.
- Values of hyperparameters⁴ used by the algorithm (number of training steps, learning rate, batch size, neural network size, neural network architecture, ...).
- The game or task being trained for.

Training agents in ML-Agents can take a lot of computation time, which limits how much data you are likely able to collect as a single group. Therefore, **we expect all groups of students to collaboratively collect and share data** as per the instructions on <https://github.com/DennisSoemers/BCS2720-project-data-collection>. It will be up to you to figure out (based on documentation all groups are expected to include with their data!) what data is useful for you, and how to effectively combine data. Trying to standardise, e.g., naming of columns is recommended.

Project week. In between periods 1 and 2, there is a *project week*. On the Friday of the Project week, you are expected to already provide us with an example of a data file you might generate, **submitted only via Canvas, not as a GitHub pull request**. This may be a small proof-of-concept file, but the general structure should be in place, such that we can provide feedback on this. Complementary documentation is also expected (this need not be long).

³<https://unity-technologies.github.io/ml-agents/Training-ML-Agents/>

⁴<https://unity-technologies.github.io/ml-agents/Training-Configuration-File/>

Assessment. In the Midway Evaluation at the end of this phase, you will give **an in-person presentation of up to 20 minutes (including at least 5 minutes for questions)** to explain and discuss your work.

2.3 Third phase

In the third phase, we expect that both data collection and ML-based analysis of the data will continue, albeit with the extent of focus between those two likely swapping around compared to phase 2. Your infrastructure for collecting data should be fully functional and be able to continue running with relatively little attention, and you can shift more attention to applying your own ML solutions.

All of your experiments should be well-documented and easily reproducible. Parameters should be changed using configuration files, or, at the bare minimum, well-documented command-line arguments. If someone wanted to reproduce your experiments (which they should be able to from the information provided in your GitHub page), it should not be necessary for them to manually change any lines of code in between experiments.

Assessment. In this phase, you will provide us with **a written report explaining your work (max 10 pages, excluding appendices and references)**. You will also give **a presentation and discuss the work with us**. A total of 30 minutes will be reserved for the presentation, and discussion, but we expect a sizeable portion of this (at least 15 minutes, preferably more) to be discussion: you only need to prepare a relatively short presentation.

References

- [1] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2020.

A Tips for Installation of Dependencies

Regardless of which operating system (OS) you use, you will require at least four of the following five dependencies. Only Visual Studio (A.2) may be unique to Windows. Some of the other tips below may also only be applicable to or precisely correct for Windows, but general recommendations will apply to, or be similar for, all OSes. You will often be able to find solutions for any trouble you run into by browsing the web.

A.1 Git

You will need to install, and become acquainted with, the `git` version control system. You will also need to create a GitHub account, and become familiar with GitHub.

A.2 Visual Studio

In principle you will not require most of Visual Studio (except if you choose to use it as your IDE of choice for any of the involved programming languages), but a part of the installation process of `ml-agents` (A.5) may require builds tools that come with Visual Studio. For Windows users, we recommend downloading Visual Studio from <https://visualstudio.microsoft.com/downloads/> (the *Community* edition). In its installer, make sure to select at least *Python development* (under *Web & Cloud*), and *Desktop development with C++* (under *Desktop & Mobile*). The following additions may also be useful, especially if during the project you end up deciding you'd like to use Visual Studio for any programming in C# for Unity:

- *.NET desktop development.*
- *Windows application development.*
- *Game development with Unity.*

A.3 Python

A.3.1 Python Versions

You will need to install Python. Make sure to pay attention to version numbers! The official documentation of `ml-agents` say that version 3.10.12 is required. However, at least for Windows users, version 3.10.11 is the latest version (among 3.10.x versions) which is still easily installable via a Windows installer, so you may wish to give this one a try anyway. Version 3.10.12 will be more challenging to install on Windows, as it must be built from source.

A.3.2 Virtual Environments

After installing Python, it is strongly recommended to create a virtual environment (using, e.g., "`<python installation directory>/python.exe -m venv <ml-agents installation directory>ml-agents/venv`", and to activate that virtual environment whenever you install any further python packages for this project, or whenever you work on this project.

A.4 Unity

Download and install the Unity game engine from <https://unity.com/>.

A.5 ml-agents

In a previous academic year (2024-2025), it was recommended *not* to clone ml-agents from the official repository, but rather from a specific branch on a fork of our own, using: `git clone --branch fix-numpy-release-21-branch https://github.com/DennisSoemers/ml-agents.git`. This was recommended because it was set up to include a fix for an important installation error reported in <https://github.com/Unity-Technologies/ml-agents/issues/6102>, which was not available from the official repository at the time. Since then, there have been updates to the official repository, so you may try if it works correctly (you may also stick to our own, older fork, if you prefer). **Make sure that all members of your group use the same version** (this should happen automatically as you should be collaborating in the same repository).